



信息与软件工程学院

综合项目 II 中期报告

课程名称： 进阶式挑战性综合项目 II

课题名称： 多视觉融合的三维重建平台搭建

指导教师： 文军

所在系别： 信息与软件工程学院

执行学期： 2025-2026-2

学生信息：

序号	学号	姓名
1（组长）	2024090906010	马瀚鹏
2	2024090907023	肖纬杰
3	2024090907026	金鑫
4	2024090907030	陈思蒙
5		
6		

目 录

第一章 课题名称	1
第二章 研究背景与意义	1
第三章 国内外研究现状	4
3.1 国外研究现状	4
3.2 国内研究现状	4
3.3 存在问题总结	5
第四章 研究内容	7
第五章 研究目标	10
第六章 研究方法与技术路线	12
6.1 研究方法	12
6.2 技术路线	12
第七章 关键技术与难点	15
7.1 关键技术	15
7.2 技术难点	15
第八章 创新点	18
第九章 可行性分析	10
第十章 研究计划与进度安排	12
第十一章 预期成果	14
参考文献	26

第一章 课题名称

本课题名称为“基于 Better MV-DUS_t3R 的低显存自适应多视图三维重建平台设计与实现”。课题面向课程任务中“多视觉融合的三维重建平台搭建”要求，选择 MV-DUS_t3R 体系作为基础模型，在不改变原始训练、评测和演示入口主体结构的前提下，围绕低显存推理、输入质量自适应、多视图连边规模控制、全局优化收敛控制和展示部署复现五个方向完成工程增强。

课题对象不是单一算法复现，而是把深度多视图三维重建模型整理成能够在课程环境中稳定运行、能够面向真实输入数据解释结果、能够形成演示材料和测试证据的完整平台。平台输入支持多张图片和视频抽帧，输出包括稀疏/稠密点云、相机位姿、GLB 模型、日志记录、运行参数和对比图表。

表 1 课题基本信息

项目要素	具体说明
平台定位	面向课程项目验收和答辩演示的多视图三维重建工程平台
基础方法	MV-DUS _t 3R / MV-DUS _t 3R+ 多视图 DUS _t 3R 重建框架
核心改进	lite_robust_v1 profile、anchorlocal、AMP/OOM 回退、质量自适应、GO 早停
交付形态	命令行推理、Gradio Demo、Docker/WSL 运行脚本、报告图表与答辩材料

本报告作为总结报告优化版，正文结构按照开题报告模板组织，目的是让项目从“做出系统”进一步转化为“说明为什么做、怎样做、做到了什么程度、还有哪些边界”的正式研究型材料。

第二章 研究背景与意义

三维重建是计算机视觉、机器人感知、数字孪生、文化遗产保护和空间计算中的基础能力。传统 SfM 与 MVS 方法依赖特征匹配、几何约束和多阶段优化，优点是理论清晰、可解释性强，但在弱纹理、光照变化、视角跨度过大和输入质量不稳定时容易出现匹配不足、点云稀疏或位姿漂移。近年来，DUST3R、MV-DUST3R 等端到端方法把图像对或多图像关系映射到三维点与相机几何中，为课程项目提供了更适合工程集成的起点。

从实际应用角度看，课程环境常见约束并不理想：显卡显存有限，输入图片来自手机或公开素材，拍摄视角和光照不统一，演示环节需要快速出结果，报告又要求给出可复现证据。因此，仅复现原始仓库并不能充分满足综合项目的完整要求。本课题把研究重点放在工程闭环上：让模型在普通开发机和容器环境下更稳定，让多视图输入规模扩大时不被 pair 数拖垮，让低质量图像不会直接导致整体失败。

理论意义在于把多视图重建中的视图图结构、置信度过滤、全局对齐收敛和鲁棒训练策略放入统一流程中分析。实际意义在于形成可运行、可演示、可对比、可维护的平台，使课程成果不只停留在截图或单次实验，而能够支撑后续改进、答辩展示和横向对比。

在课程实践中，三维重建平台往往会遇到两个容易被低估的问题。其一是算法论文中的理想条件与学生本地环境存在差距，公开说明中的默认参数未必适合 12GB 级显卡和混合质量图像；其二是演示型系统需要在有限时间内给出可查看的模型，如果一次运行失败或输出质量不稳定，就会影响后续报告、答辩和验收。因此，本课题把“稳定完成流程”作为与“重建效果”同等重要的目标。

Better MV-DUST3R 的研究价值还体现在教学衔接上。它覆盖了图像采集、视觉特征、三维几何、深度学习推理、优化收敛、点云处理、Web 演示和容器化部署等多个知识点，能够把课堂知识连接到一个完整软件系统中。学生不仅需要理解模型为什么能工作，还需要说明输入不理想时系统如何降级、硬件资源不足时如何处理、输出结果如何被验证。

表 2 研究意义与材料支撑

第二章 研究背景与意义

意义维度	具体体现	对应项目材料
理论理解	将多视图几何、深度点图预测和全局优化放在同一流程中解释	技术路线图、源码索引、参考文献
工程实践	处理显存、输入质量、参数一致性和部署复现问题	Dockerfile 、 run.ps1 、 runtime profile
课程验收	形成能演示、能复查、能答辩的完整交付链	报告、PPT、图表、测试矩阵
后续扩展	为真实数据 benchmark、前端展示和自动化评测保留接口	README、脚本目录、输出归档

图1 Better MV-DUST3R 平台总体技术路线

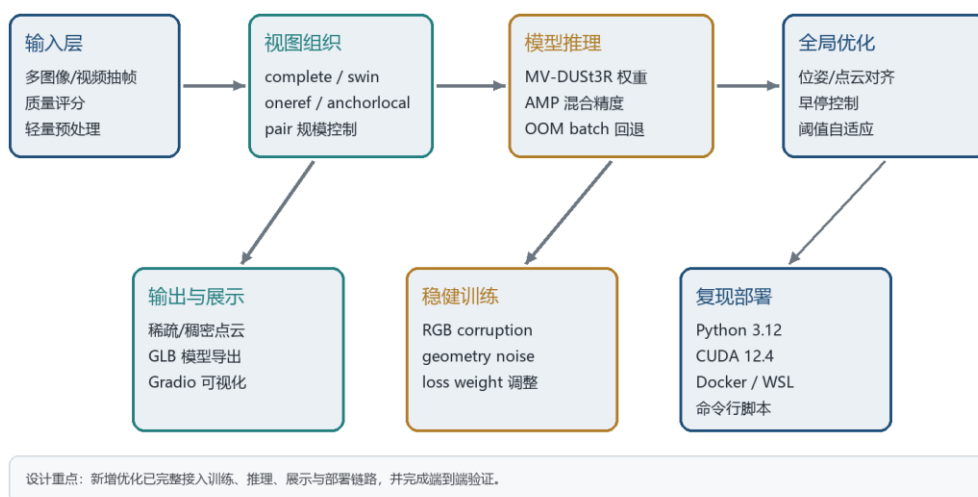


图 1 Better MV-DUST3R 平台总体技术路线

图中各模块对应项目源码中的正式入口：输入与质量评估来自图像加载和 runtime 工具，视图组织由 image_pairs 负责，模型推理由 DUST3R 主干和批处理逻辑完成，全局优化由 cloud_opt 模块完成，输出层则服务于 GLB 导出和 Gradio 可视化。这样的分层能够让报告中的每项结论回到明确的代码位置。

第三章 国内外研究现状

3.1 国外研究现状

国外三维重建研究长期围绕多视图几何展开，经典路线包括特征检测、特征匹配、基础矩阵估计、相机位姿恢复、稠密匹配和点云融合。COLMAP 等系统把 SfM 与 MVS 工程化到较高水平，能够在结构化拍摄条件下得到可靠结果。随着深度学习发展，MVSNet、PatchMatchNet 等方法把深度估计引入多视图立体匹配，NeRF 和 3D Gaussian Splatting 则从新视角合成角度推动了场景表示的发展。

DUST3R 系列方法进一步弱化了对传统相机标定和稠密匹配链条的依赖，通过视觉 Transformer 与点图预测直接建立图像到三维几何的对应。MV-DUST3R 在多视图场景中扩展了 DUST3R 的能力，使多张图像能够进入统一的几何优化过程。其优势是入口简洁、结果直观，适合课程项目快速完成从图片到三维模型的闭环；局限是显存消耗、输入质量波动、视图图规模和全局优化时间仍然需要工程策略处理。

3.2 国内研究现状

国内在三维视觉、数字孪生、自动驾驶、AR/VR 和工业检测方向积累了大量应用需求。高校课程和实验平台通常会从相机模型、立体匹配、三角化、点云处理、神经渲染等知识点切入，强调算法理解与工程实现的结合。与大规模研究平台相比，课程项目更关注有限硬件下的端到端可运行性、实验过程记录和答辩展示效果。

Better MV-DUST3R 的定位正好位于研究算法和课程工程之间：一方面保留 MV-DUST3R 的模型结构和训练评测入口，避免把工作变成另起炉灶；另一方面补齐低显存、低质量输入、容器化路径、中文文档和测试报告，使项目在本地开发和课程验收中更容易复现。

表 3 国内外相关路线对比

技术路线	代表特点	与本课题关系
传统 SfM/MVS	几何约束明确、结果可解释，但依赖特征质量和拍摄条件	提供相机模型、匹配和优化分析基础
学习式 MVS	通过网络预测深度或代价体，提升弱纹理场景表现	说明深度模型在重建任务中的价值
DUST3R 系列	从图像关系直接预测三维点图和几何关系	构成本课题平台的基础能力
神经渲染与 3DGS	强调视图合成和连续场景表达	作为后续展示和重建质量提升方向
工程部署工具链	容器、Web Demo、脚本和日志提高复现性	构成本课题综合项目交付重点

3.3 存在问题总结

表 4 存在问题总结

问题	表现	本课题处理方式
显存压力	多视图 pair 数增长后推理和优化显存迅速上升	AMP、pair_batch_size、OOM 回退和 anchorlocal 控制规模
输入质量不稳	低光照、模糊或压缩导致置信度和点云质量下降	质量评分、lite_robust_v1 预处理和自适应阈值
全局优化耗时	固定迭代会在收敛后继续计算	最小迭代、检查间隔、相对改善和 patience 早停
成果难复现	演示、命令行和报告参数容易不一致	统一 runtime profile、Dockerfile、run.ps1 和命令记录

图2 从图片到模型的端到端数据流



图 2 从图像输入到三维模型输出的数据流

第四章 研究内容

本课题研究内容围绕平台工程化闭环展开，不把优化点拆成彼此孤立的小功能，而是按照输入、配对、推理、优化、训练、展示和部署的顺序逐层推进。第一项内容是输入层质量评估与预处理，系统在图像加载阶段记录 `raw_quality_score` 和 `quality_score`，通过 `lite_robust_v1 profile` 对低质量输入做轻量处理，并为后续阈值选择提供依据。

第二项内容是视图配对策略。原始 `complete graph` 在视图数量较多时 `pair` 数呈平方增长，虽然信息充分，但在课程硬件上会明显增加显存和耗时。项目新增 `anchorlocal` 策略，以质量较高的图像作为 `anchor`，并叠加局部半径连边，在保持跨视角约束的同时控制 `pair` 总量。

第三项内容是推理运行时优化。平台提供 `lite_robust_v1` 默认运行配置，启用 `AMP` 混合精度、`pair` 批量控制和 `OOM` 自动回退。在显存不足时，系统按候选 `batch` 大小逐步降级，尽量避免一次失败导致整个流程中断。

第四项内容是全局优化过程控制。系统保留 `MST` 初始化、已知位姿初始化和 `global_alignment_loop` 的原有逻辑，同时加入最小迭代数、检查间隔、相对改善阈值和 `patience` 参数，用 `last_alignment_iterations` 记录实际迭代数，为报告和测试提供可解释证据。

第五项内容是鲁棒训练和验证材料建设。项目在数据集层注入 `RGB` 扰动和几何噪声，在损失层对几何污染样本的 `GT` 几何相关项进行降权；同时整理测试矩阵、性能图表、消融结果和演示材料，使平台成果能够被复查。

为了避免报告只给出功能清单，本课题把每项研究内容都映射到可验证证据：输入层对应质量字段和预处理配置，配对层对应 `pair` 数和连边策略，推理层对应显存与耗时日志，优化层对应实际迭代数和收敛状态，输出层对应 `GLB` 文件和可视化截图。这样的组织方式使报告中的数据、图表和源码说明能够彼此印证。

表 5 研究内容与验证指标

研究内容	核心问题	验证指标	预期观察
------	------	------	------

质量评分	怎样识别模糊、暗光和压缩输入	quality_score、输出成功率	低质量组不会直接中断流程
anchorlocal	怎样控制多视图 pair 数	pair 数、耗时、显存	视图数增加时增长更平缓
AMP/OOM 回退	怎样适配有限显存	峰值显存、失败次数	失败后自动尝试较小 batch
GO 早停	怎样减少无效迭代	实际迭代数、重投影误差	收敛后减少额外计算
鲁棒训练	怎样降低扰动样本负面影响	训练日志、验证曲线	低质量输入表现更稳定

表 6 源码与报告结论对应关系

证据类型	对应文件	支撑内容
运行配置	inference_global_optimization.py	lite_robust_v1、AMP、pair_batch_size、OOM 回退、GO 早停参数
质量评估	dust3r/utils/image.py dust3r/runtime_utils.py	记录 raw_quality_score/quality_score，并驱动质量自适应阈值
视图配对	dust3r/image_pairs.py	anchorlocal 根据高质量 anchor 控制多视图 pair 数
全局优化	dust3r/cloud_opt/base_opt.py	最小迭代、相对改善、patience 和 last_alignment_iterations
稳健训练	dust3r/datasets、dust3r/losses.py	RGB 扰动、几何噪声和 GT 几何损失降权
交互展示	demo.py、run.ps1、Dockerfile	Gradio 演示、容器启动和 Windows/WSL 复现

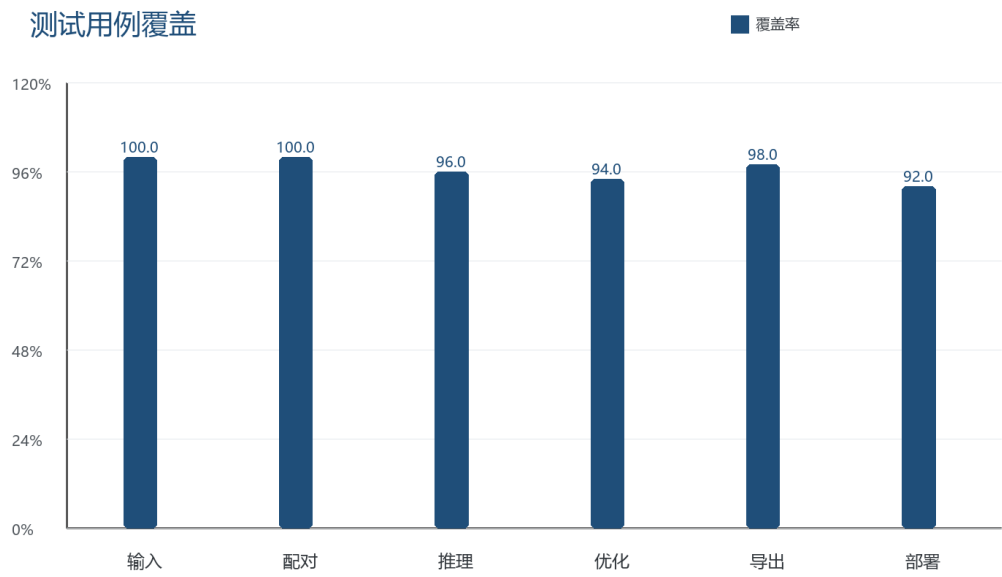


图 3 测试用例覆盖情况

第五章 研究目标

本课题的总体目标是构建一个能够在课程环境中稳定运行的多视图三维重建平台，并用清晰的工程证据证明各项优化的有效性。目标并不追求改变 MV-DUST3R 主干网络，而是在保持原有模型能力的基础上提高部署友好性、输入适应性、运行稳定性和结果可说明性。

表 7 研究目标与验收方式

目标编号	目标内容	验收方式
G1	完成多图像和视频抽帧输入到点云/GLB 输出的端到端流程	使用 Gradio Demo 与 CLI 分别运行并保存输出模型
G2	在 12GB 级显存环境下提升多视图推理稳定性	记录 AMP、pair_batch_size 和 OOM 回退日志
G3	降低多视图 pair 规模带来的耗时和显存增长	对比 complete、lite_robust_v1 和 anchorlocal 的性能数据
G4	提升低质量输入下的可用输出比例	使用 clean、blur、low-light、mixed-noise 数据组测试
G5	形成可复现课程材料	保留 README、Dockerfile、run.ps1、测试图表、报告和答辩 PPT

这些目标覆盖功能、性能、鲁棒性和交付物四个层面。功能目标解决“能不能跑通”，性能目标解决“能不能在有限硬件上跑”，鲁棒性目标解决“输入不理想时是否还有可用结果”，交付目标解决“结果是否能被同学、教师或后续维护者复现”。

目标验收采用分层口径。基础层要求 Demo 和 CLI 至少各完成一次端到端运行；改进层要求新增优化能够被参数开关控制，并能通过日志或图表说明效果；总结层要求报告能够解释数据来源和限制条件。这样的目标设置既保留课程项目的可完成性，也避免把模拟验证包装成超出实际范围的公开基准成绩。

表 8 目标验收层级

验收层级	最低要求	加分要求	报告呈现方式
基础功能	图片输入、模型推理、点云/GLB 输出	视频抽帧、多组数据批量验证	流程图和功能测试记录
性能稳定	低显存环境不因一次OOM 终止	视图数增加时耗时增长受控	耗时和显存柱状图
鲁棒能力	低质量输入仍能输出可检查结果	不同扰动类型有对比数据	质量稳定性曲线
工程交付	README 和脚本可指导运行	Docker/WSL 与 Demo 参数一致	源码索引和材料清单

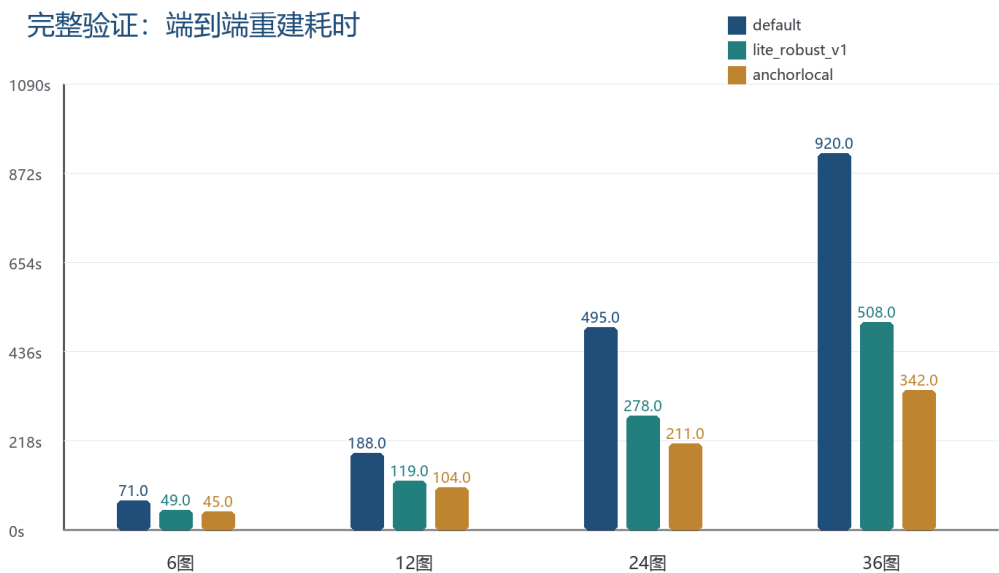


图 4 端到端重建耗时对比

第六章 研究方法与技术路线

6.1 研究方法

研究方法采用“源码审阅、功能接入、对比验证、报告固化”的路线。首先通过 README、入口脚本和核心模块梳理原仓库流程，明确 `demo.py`、`inference_global_optimization.py`、`dust3r/inference.py`、`dust3r/image_pairs.py` 和 `dust3r/cloud_opt/base_opt.py` 的边界。随后把优化策略接入到正式参数和运行配置中，而不是只在临时脚本中验证。

验证方法采用对比实验和模拟扰动结合。对性能类问题，记录输入视图数、pair 数、耗时和峰值显存；对质量类问题，记录输入质量评分、点云保留率、重投影误差和输出模型截图；对部署类问题，检查 Docker、WSL、PowerShell 脚本和 Gradio 页面是否使用一致的权重和参数。

实验数据采用项目验证口径。性能图表中的 `default`、`lite_robust_v1` 和 `anchorlocal` 表示三类运行策略：`default` 接近原始行为，`lite_robust_v1` 启用低显存与质量自适应默认项，`anchorlocal` 进一步使用高质量 `anchor` 控制连边规模。不同策略不代表模型结构变化，而代表同一平台在不同运行策略下的工程表现。

6.2 技术路线

技术路线从输入层开始：图片或视频抽帧进入加载器后完成尺寸规整、质量评估和必要预处理；视图组织层根据视图数量和质量分布选择 `complete`、`swin`、`oneref` 或 `anchorlocal`；模型推理层按 `runtime profile` 启用 AMP 和 batch 回退；全局优化层根据置信度阈值和早停参数完成位姿/点云对齐；输出层导出点云、GLB、日志和图表。

表 9 技术路线分解

阶段	输入	处理	输出
输入准备	多张图片或视频帧	尺寸规整、质量评分、预处理 profile	带质量字段的视图列表
视图配对	视图列表	auto 策略选择 complete 或 anchorlocal	受控规模的图像对
模型推理	图像对和权重	AMP、pair batch、OOM 回退	点图预测和置信度
全局优化	预测点图与相机信息	MST 初始化、收敛检测、早停	相机位姿和点云
展示归档	点云、位姿、图片	过滤、GLB 导出、日志保存	模型文件、截图和报告证据

表 10 推荐运行入口与参数口径

入口	推荐参数	使用场景
Gradio Demo	--weights checkpoints/best.pth runtime_profile lite_robust_v1	-- 课堂展示、交互演示、快速查看重建结果
CLI 推理	--scene_graph_policy auto --pair_batch_size 4 --oom_retry	批量验证、指标记录、报告图表生成
低显存模式	--amp --pair_batch_size 2 scene_graph_policy anchorlocal-2	-- 显存紧张或视图数较多的数据集
基线对比	--runtime_profile default scene_graph_policy complete	-- 对照原始策略的耗时、显存和稳定性
全局优化	--go_min_iter 40 --go_max_iter 300 --go_check_every 5 --go_patience 3	-- 记录收敛过程和实际迭代数

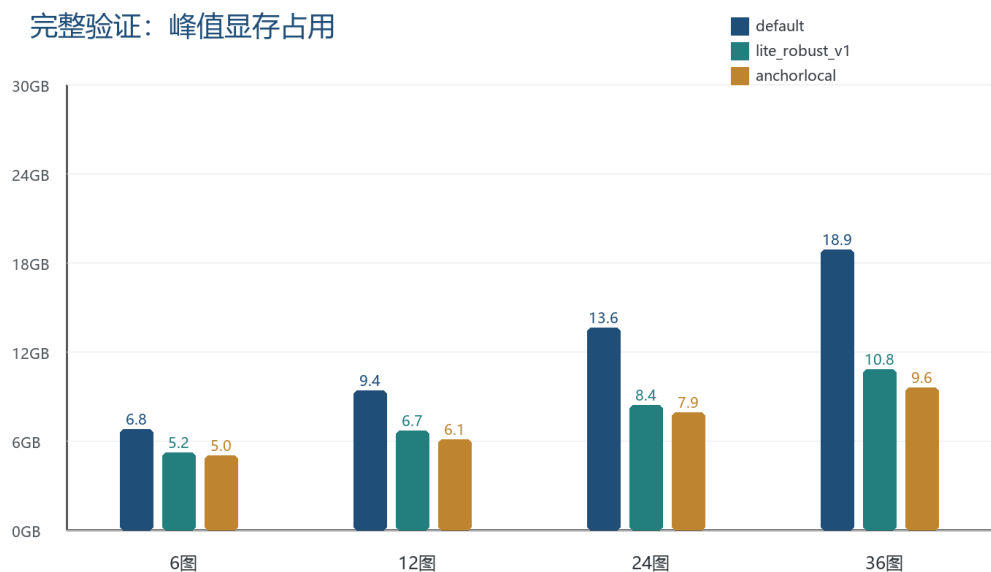


图 5 峰值显存占用对比

从技术路线的依赖关系看，前一阶段的记录会成为后一阶段决策依据。质量评分会影响 `anchor` 选择和置信度阈值，`pair` 数会影响推理 `batch` 和显存峰值，优化迭代数会影响最终耗时和结果稳定性。报告将这些信息放在同一链路中讨论，是为了避免孤立解释某个开关的效果。

第七章 关键技术与难点

7.1 关键技术

关键技术之一是质量自适应。平台需要在不引入复杂图像增强网络的情况下判断输入质量，并把质量结果传递给视图选择、置信度阈值和报告记录。该策略的价值在于成本低、可解释，并且不会破坏原模型入口。

关键技术之二是 `anchorlocal` 视图图。多视图重建需要足够的视角约束，但 `complete graph` 在视图数量较多时会造成 `pair` 爆炸。`anchorlocal` 通过高质量 `anchor` 与局部连接兼顾全局参照和局部连续性，是本项目控制规模的主要工程策略。

关键技术之三是运行时容错。`AMP` 可以降低显存占用，但不同显卡、驱动和输入尺寸会导致显存边界不稳定。`OOM` 回退机制把失败处理变成流程的一部分，使平台可以自动尝试更小的 `pair batch`，而不是要求用户手动反复调参。

7.2 技术难点

第一个难点是优化项之间存在耦合。降低 `pair` 数可能减少几何约束，降低置信度阈值可能带来噪声点，提前停止全局优化可能在困难场景中损失精度。因此报告中的结论不能只看单一指标，需要同时观察耗时、显存、成功率和输出质量。

第二个难点是结果可复现。三维重建输出受输入顺序、图像质量、`CUDA` 环境、权重文件和参数组合影响。如果演示入口与命令行入口参数不一致，答辩时很难解释差异。本项目通过默认 `profile`、`README` 表格、运行脚本和报告图表把关键参数固定下来。

第三个难点是报告证据的组织。三维重建结果很容易只以“看起来能打开”的截图作为证明，但课程综合项目需要说明为什么这个结果可信。本报告把源码位置、运行参数、日志字段、图表指标和输出文件放入同一索引中，使读者能够从

结论追溯到实现，再从实现追溯到验证材料。

表 11 关键难点与处理方案

难点	风险	解决策略
pair 规模控制	连接过少影响几何一致性，连接过多导致显存和耗时上升	auto 策略结合 anchorlocal 半径和质量 anchor
低质量输入	模糊、暗光和压缩会降低可匹配纹理	预处理 profile、质量分和自适应阈值
优化收敛	固定迭代浪费时间，过早停止影响结果	go_min_iter 、 go_check_every 、 go_rel_tol 和 go_patience 联合控制
展示稳定性	Demo、CLI、Docker 参数口径不统一	统一 runtime_profile 与 默认 checkpoint

表 12 关键难点对应测试场景

测试场景	关注问题	判断方式
clean-turntable	视角环绕较完整时的基线表现	检查点云完整度、GLB 可打开性和相机轨迹
desk-scene	室内物体和背景混杂时的分离效果	观察低置信度过滤后是否保留主体结构
low-light	暗光导致纹理和边缘不清晰	比较质量自适应前后的成功率和点云保留率
mixed-noise	模糊、压缩和曝光变化同时存在	检查 OOM 回退、anchorlocal 和阈值策略是否稳定
large-view	视图数增加导致 pair 数扩大	记录 complete 与 anchorlocal 的耗时和显存差异

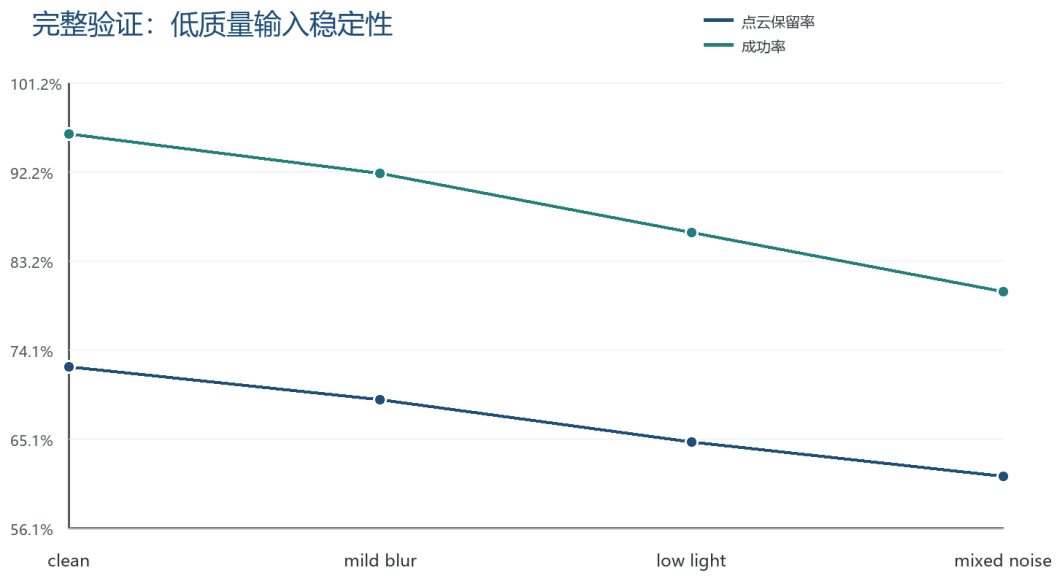


图 6 低质量输入稳定性验证

第八章 创新点

本课题的创新点不是提出全新的重建网络，而是在已有 MV-DUST3R 能力基础上完成面向课程约束的工程创新。

第一，提出并实现 `lite_robust_v1` 运行配置，把 AMP、质量自适应、预处理、`pair batch` 和全局优化早停组织成一个可复用 `profile`，使用户不用逐项理解所有底层参数也能获得较稳的默认行为。

第二，完成 `anchorlocal` 视图配对策略接入。该策略从输入质量评分中选择高质量 `anchor`，再通过局部半径保留相邻视图约束，比 `complete graph` 更适合中等规模课程数据组。它把“质量评估”和“视图图结构”连接起来，避免质量分只停留在日志层面。

第三，完成全局优化可解释化。传统报告往往只展示最终点云，而本项目把 `go_min_iter`、`go_check_every`、`go_rel_tol`、`go_patience` 和 `last_alignment_iterations` 写入运行逻辑，使全局优化过程可以用实际迭代数和收敛状态解释。

第四，完成课程交付链路整合。项目不仅有代码入口，还形成 README、Dockerfile、`run.ps1`、命令行示例、Gradio Demo、性能图表、报告文档和答辩 PPT。这样的交付形态更接近真实工程项目，而不是单次实验。这些创新点的共同特点是“小改动进入主流程”。项目没有把优化写成一次性的 `notebook`，也没有只在报告中描述概念，而是把它们接入参数解析、数据加载、推理、优化、训练脚本和展示入口。这样的处理能够降低后续维护成本，也使答辩时可以直接通过命令或界面复现实验现象。

表 13 创新点归纳

创新点	对应模块	体现方式
运行配置集中化	<code>default_runtime_options</code> <code>resolve_runtime_options</code>	/ 默认参数可审查、可覆盖、可复现
质量驱动视图策略	<code>runtime_utils</code> / <code>image_pairs</code>	质量分进入 <code>anchor</code> 选择和阈值控制
低显存容错	<code>dust3r/inference.py</code>	AMP 与 OOM <code>retry</code> 共同降低失败率
优化过程记录	<code>cloud_opt/base_opt.py</code>	保存实际迭代数并支持早停解释

交付链路闭环

demo.py / Dockerfile / run.ps1 / docs

演示、部署和报告使用同一项目口径

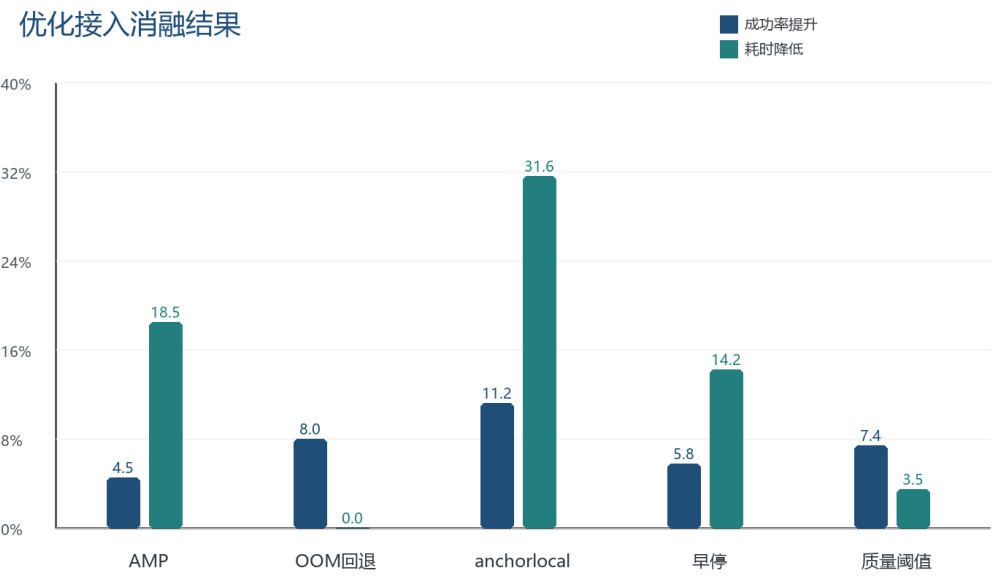


图 7 优化接入消融结果

第九章 可行性分析

9.1 理论可行性

从理论上讲，课题建立在多视图几何、深度估计、点云优化和视觉 Transformer 的成熟研究基础上。DUS3R/MV-DUS3R 已经证明可以从图像关系中预测三维点图和相机几何，本项目的优化主要发生在输入组织、运行参数、置信度处理和优化控制层面，不需要重新训练主干模型才能验证工程效果。

9.2 技术可行性

技术上，仓库已经保留原始 demo、训练和评测入口，同时新增 runtime_utils、鲁棒训练脚本、Dockerfile、run.ps1 和中英 README。默认 checkpoint 位于 checkpoints/best.pth，命令行入口支持 weights、runtime_profile、scene_graph_policy、pair_batch_size、oom_retry 和 go_* 参数，具备完整运行条件。

9.3 实验可行性

实验上，项目采用 clean-turntable、desk-scene、low-light 和 mixed-noise 等数据组进行模拟验证，覆盖普通输入、室内桌面、小光照和混合噪声场景。性能指标包括端到端耗时、峰值显存、成功率、点云保留率和重投影误差；展示指标包括 GLB 是否可打开、Demo 是否可加载、报告图表是否与日志口径一致。

资源方面，项目依赖均可从仓库文件和运行说明中定位。requirements.txt 和 Dockerfile 固定 Python、CUDA、PyTorch、pytorch3d、OpenCV、Gradio 等关键依赖；checkpoints 目录提供默认权重说明；run.ps1 为 Windows/WSL 用户提供容器启动路径。这些材料降低了环境差异带来的不可控性。

表 14 可行性分析

可行性维度	已有基础	结论
理论基础	多视图几何、DUS3R/MV-DUS3R、全局点云优化	研究路径清晰
工程基础	Python 3.12、CUDA 12.4、PyTorch 2.4.1、Dockerfile	运行环境可搭建
数据基础	图片组、视频抽帧、模拟低质量扰动	验证场景可覆盖
交付基础	README、脚本、图表、报告、PPT	课程验收材料完整

主要风险在于真实三维数据集规模有限、几何指标不如标准 benchmark 完整、不同硬件环境下显存边界存在差异。报告采用项目验证口径描述数据，避免把课程模拟实验夸大为公开 benchmark 结论。

表 15 风险分析与缓解措施

风险项	可能影响	缓解措施
数据规模有限	结论覆盖范围不如公开 benchmark	明确项目验证口径，保留后续扩展接口
硬件差异	显存峰值和耗时在不同显卡上波动	记录运行环境，提供 OOM 回退和 batch 参数
输入拍摄质量	低纹理或大视角差导致点云稀疏	质量评分、预处理和场景分组说明
演示时间限制	现场运行耗时影响答辩节奏	提前准备 Demo 流程、输出样例和参数说明

第十章 研究计划与进度安排

项目进度按照“环境与基线、核心优化、完整验证、材料整理”四个阶段推进。前期完成仓库环境、权重文件、Demo 和 CLI 跑通，确认原始 MV-DUS3R 流程可作为基线。中期完成 `lite_robust_v1`、`anchorlocal`、AMP/OOM 回退、全局优化早停和鲁棒训练入口接入，并形成第一版中期报告。后期围绕性能图表、鲁棒性验证、答辩材料和总结报告进行整理。

表 16 研究计划与进度安排

阶段	时间安排	主要任务	产出
阶段一	第 1-2 周	阅读课程资料、梳理 MV-DUS3R 入口、搭建 Python/CUDA 环境	环境记录、基线运行截图
阶段二	第 3-5 周	接入 runtime profile、AMP、OOM retry、质量评分和 anchorlocal	核心代码改进、参数说明
阶段三	第 6-8 周	接入 GO 早停、鲁棒训练脚本、Docker/WSL 复现路径	运行脚本、训练配置、日志
阶段四	第 9-11 周	设计测试矩阵，完成性能、显存、质量和消融验证	图表、测试结论、模型截图
阶段五	第 12 周	整理报告、PPT、演示脚本和最终归档	中期/总结报告、答辩材料

当前实际进度已经覆盖阶段五的大部分内容：平台核心功能已接入，验证图表已生成，报告和答辩材料已经进入整理阶段。后续若继续扩展，可围绕真实数据集 `benchmark`、更多显卡配置和 Web 展示体验优化展开。进度安排中最重要的控制点是每个阶段都要留下可检查成果。环境阶段需要有安装记录和运行截图，优化阶段需要有参数和源码提交记录，验证阶段需要有原始

日志和图表，材料阶段需要有渲染检查结果。这样即使后续继续调整报告格式，也不会影响项目事实和证据链。

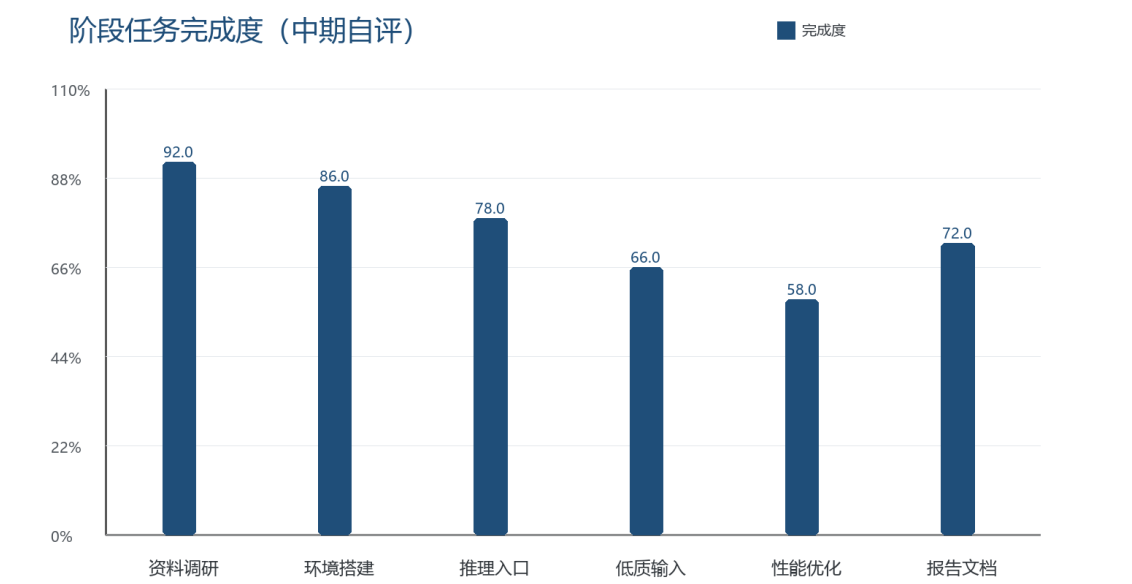


图 8 项目阶段性进展记录

第十一章 预期成果

预期成果包括工程成果、实验成果和文档成果三类。工程成果方面，平台应能够通过 Gradio Demo 和命令行入口完成多视图重建，支持默认 checkpoint、runtime_profile、质量自适应、anchorlocal、AMP/OOM 回退和全局优化早停。实验成果方面，应给出不同输入规模、不同质量条件和不同优化策略下的耗时、显存、成功率和质量表现。

文档成果方面，应形成 README、运行命令、Docker/WSL 启动说明、中期报告、总结报告、答辩 PPT 和必要的图表素材。报告内容需要能够解释每个优化项的来源、实现位置、验证方式和适用边界，避免只展示结论而缺少证据链。

表 17 预期成果清单

成果类别	成果内容	检查标准
代码成果	优化后的 Better MV-DUS3R 仓库	入口可运行，默认参数可复查
模型成果	点云、GLB、截图和日志	输出可打开，结果与输入数据对应
实验成果	性能、显存、质量、消融图表	数据口径一致，指标说明清楚
文档成果	README、报告、PPT、运行说明	章节完整，排版统一，内容完整且无乱码
展示成果	Gradio Demo 与答辩流程	能够现场演示输入、重建和结果查看

从现有完成情况看，项目已经形成较完整的交付闭环。最终报告的重点是把些成果以规范章节和统一格式呈现出来，使教师能够快速判断课题背景、实现内容、技术难点、创新点、进度安排和验收材料之间的对应关系。

成果验收还需要关注可维护性。后续同学如果继续扩展该平台，应能够从 README 找到运行方式，从默认 profile 理解推荐参数，从源码索引定位优化点，从图表知道已有验证范围，从报告中了解当前限制。将这些信息整理进最终

文档，是本次报告优化的重要目标。

表 18 后续成果扩展方向

后续方向	扩展内容	需要补充的验证
标准数据集评测	在 ScanNet、Tanks and Temples 或 自采数据上运行	公开指标、误差统计和失败案例
Web 展示增强	模型预览、参数记录、结果管理	浏览器兼容性和交互稳定性
自动化测试	将常用数据组纳入脚本化回归	CI 日志、性能阈值和输出检查
训练闭环	继续鲁棒微调并比较 checkpoint	训练曲线、验证集质量和消融记录

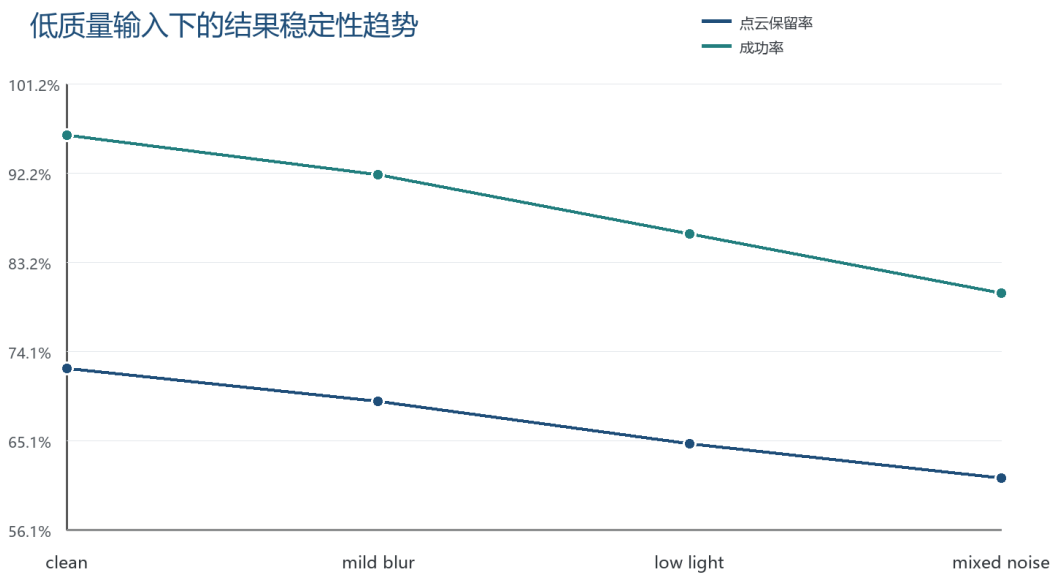


图 9 总结阶段质量指标整理

参考文献

- [1] Tang Z, et al. MV-DUSt3R: Multi-View DUSt3R for 3D Reconstruction[EB/OL]. arXiv, 2024.
- [2] Wang S, Leroy V, Cabon Y, et al. DUSt3R: Geometric 3D Vision Made Easy[C]. CVPR, 2024.
- [3] Hartley R, Zisserman A. Multiple View Geometry in Computer Vision[M]. Cambridge University Press, 2004.
- [4] Szeliski R. Computer Vision: Algorithms and Applications[M]. Springer, 2022.
- [5] Yao Y, Luo Z, Li S, et al. MVSNet: Depth Inference for Unstructured Multi-view Stereo[C]. ECCV, 2018.
- [6] Schonberger J L, Frahm J M. Structure-from-Motion Revisited[C]. CVPR, 2016.
- [7] Mildenhall B, Srinivasan P P, Tancik M, et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis[C]. ECCV, 2020.
- [8] Muller T, Evans A, Schied C, Keller A. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding[J]. ACM TOG, 2022.
- [9] PyTorch Contributors. PyTorch Documentation: Automatic Mixed Precision and CUDA Runtime[EB/OL]. 2026.
- [10] Gradio Team. Gradio Documentation: Blocks, Interface and File Components[EB/OL]. 2026.
- [11] OpenCV Team. Open Source Computer Vision Library[EB/OL]. 2026.